Module 06 Lab - Regression and Classification Models**Objective:** To understand the difference between regression and classification and to build your first linear models for both tasks.**In this lab, you will write the code to train and evaluate the models.**

Part 1: Regression vs. ClassificationThis is the most fundamental distinction between types of supervised learning problems.* **Regression:** The goal is to predict a **continuous numerical value**. * *Examples:* Predicting the price of a house, the temperature tomorrow, or the stock price.* **Classification:** The goal is to predict a **discrete category or class label**. * *Examples:* Predicting if an email is spam or not spam, if a flower is a setosa, versicolor, or virginica, or if a customer will churn or not.**In this lab, we will tackle one of each.**

Part 2: Linear Regression**Concept:** Linear Regression is used to predict a continuous value. It works by finding the best-fitting straight line through the data points. The model learns a "slope" (coefficient) for each feature and an "intercept".* **Problem:** We will predict the `Fare` of a Titanic passenger based on their `Age` and `Pclass`.

```
import pandas as pd
from sklearn.model_selection import train_test_split

# Load and prepare the data
df = pd.read_csv('https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv')

# Drop rows with missing Age
df.dropna(subset=['Age'], inplace=True)

# Define features and target
features = ['Age', 'Pclass']
target_reg = 'Fare'

X_reg = df[features]
y_reg = df[target_reg]

# Split the data
X_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(
    X_reg, y_reg, test_size=0.2, random_state=42
)
```

Task 1: Train and Evaluate a Linear Regression Model**Your Task:**1. Create an instance of the `LinearRegression` model.2. Train the model using the training data (`X_train_reg`, `y_train_reg`).3. Make predictions on the test data.4. Evaluate the model using `mean_squared_error`. This metric tells us the average of the squared differences between the predicted and actual values.

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# Create the model
lin_reg = LinearRegression()

# Train the model
lin_reg.fit(X_train_reg, y_train_reg)

# Make predictions
y_pred_reg = lin_reg.predict(X_test_reg)
```

```
# Evaluate
mse = mean_squared_error(y_test_reg, y_pred_reg)
print("Mean Squared Error:", mse)
```

```
Mean Squared Error: 3364.922912856981
```

Part 3: Logistic Regression**Concept:** Despite its name, Logistic Regression is a **classification** algorithm. It works by calculating the probability that a given input belongs to a certain class. It's one of the most widely used and interpretable classification models.* **Problem:** We will predict whether a passenger `Survived` based on their `Age`, `Pclass`, and `Sex`.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report

# Load dataset
df = pd.read_csv('https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv')

# Select relevant columns and drop missing values
df = df[['Survived', 'Age', 'Pclass', 'Sex']].dropna()

# One-hot encode Sex (male/female)
df = pd.get_dummies(df, columns=['Sex'], drop_first=True)

# Define features and target
features = ['Age', 'Pclass', 'Sex_male']
target = 'Survived'

X = df[features]
y = df[target]

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Task 2: Train and Evaluate a Logistic Regression Model**Your Task:**1. Create an instance of the `LogisticRegression` model.2. Train the model using the classification training data.3. Make predictions on the test data.4. Evaluate the model using `accuracy_score`.

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# 1. Create an instance of the Logistic Regression model
log_reg = LogisticRegression(max_iter=1000)

# 2. Train the model using the classification training data
log_reg.fit(X_train, y_train)

# 3. Make predictions on the test data
y_pred = log_reg.predict(X_test)

# 4. Evaluate the model using accuracy_score
accuracy = accuracy_score(y_test, y_pred)
print("Model Accuracy:", accuracy)
```

```
Model Accuracy: 0.7482517482517482
```

📝 Knowledge Check**Instructions:** Answer the following questions in this markdown cell.1. **In your own words, what is the key difference between a regression problem and a classification problem?**2. **The `LinearRegression` model has an attribute called `.coef_`. After you**

train the model, print `lr_model.coef_`. What do these numbers represent?3. Why did we use `mean_squared_error` to evaluate the regression model but `accuracy_score` for the classification model? Why wouldn't accuracy be a good metric for the fare prediction task?[ENTER YOUR ANSWERS HERE]

1. **In your own words, what is the key difference between a regression problem and a classification problem?** The key difference is the type of output being predicted. Regression problems predict a continuous numerical value, such as a fare or a price, while classification problems predict a discrete category or label, such as survived vs not survived.
2. **The LinearRegression model has an attribute called .coef_. After you train the model, print lr_model.coef_. What do these numbers represent?** The values in .coef_ represent how much each input feature contributes to the prediction. Each coefficient shows how much the predicted fare changes when that feature increases by one unit, assuming the other features stay the same.
3. **Why did we use mean_squared_error to evaluate the regression model but accuracy_score for the classification model? Why wouldn't accuracy be a good metric for the fare prediction task?** Mean squared error is used for regression because it measures how far the predicted numerical values are from the actual values. Accuracy works for classification because predictions are either correct or incorrect. Accuracy would not make sense for fare prediction since fares are continuous values and there is no clear definition of a prediction being simply “right” or “wrong.”